ELF-DOS User's Guide

ELF-DOS is a disk operating system for the CDP1802 microprocessor. It uses the FAT file system, the same one used by early versions of MS-DOS, so disks made on ELF-DOS can be read on most other computers, and vice versa.

This guide explains how to use ELF-DOS from day to day: starting the computer, typing commands, working with files and directories, and writing batch files. If you want to know how ELF-DOS works on the inside, or how to write your own programs for it, see the *ELF-DOS Developer's Guide*.

Starting ELF-DOS

To put ELF-DOS on a card in the first place, see the installation guide for your system: `INSTALL-linux.md` or `INSTALL-windows.md` (`INSTALL.md` in a release package).

When you turn on the computer, ELF-DOS prints a short banner and shows you a prompt, such as:

```
C:/>
```

The letter before the colon is the current drive. At start-up ELF-DOS reads the partition table of the drive it booted from and makes each of its partitions available, lettered `C:` through `F:` in order. The text after the colon is the current directory on that drive. If your current directory is more than one level deep from the root, the first part of the path is replaced by `...`; use the `PWD` command to see the full path.

Those four are only a starting point. Six drives can be in use at once, and a drive can carry any letter from `A:` to `Z:` - see "Drives" below.

To switch to a different drive, type its letter followed by a colon, and press Enter:

```
D:
```

The `CD` command changes the current directory. Typing `CD` by itself with just a drive letter (`CD D:`) does not switch you to that drive. It shows or changes D:'s own current directory without leaving the drive you are on. To actually change drives, type the drive letter alone as shown above.

If a file named `autoexec.bat` exists in the root directory of the boot drive, ELF-DOS runs it automatically, before showing you the first prompt. See "Writing Batch Files" below for what a batch file is and how to write one.

Drives

The four drives set up at start-up are not fixed. The `MOUNT` command attaches any partition to any letter, and `UMOUNT` detaches one. Typing `MOUNT` on its own lists what is currently in use:

```
C:/> MOUNT
Drive  Unit  Partition start LBA
C:     0     2,048
D:     0     1,955,840
E:     0     4,052,992
F:     0     6,150,144
4 of 6 drives mounted.
```

Six drives can be mounted at once, and a letter is just a name: nothing stops you from calling a partition `W:` because it holds your work, or `M:` for music. `MOUNT` takes the device's unit number, the partition number, and the letter. To attach the second partition on unit 0 as `W:`:

```
C:/> MOUNT 0 2 W:
Mounted partition 2 as W:
```

Mounting a letter that is already in use simply re-points it. To free a slot, unmount it:

```
C:/> UMOUNT W:
Unmounted W:
```

If you unmount the drive you are currently on, you are moved back to the drive ELF-DOS booted from.

Other devices. Some machines have more than one storage device, and the unit number says which one: `MOUNT 1 2 W:` means "unit 1, partition 2, as `W:`". Most machines have only unit 0. The unit is always required, so a `MOUNT` line means the same device whichever one the machine booted from; `MOUNT` lists each drive's unit, including the drives ELF-DOS set up from the boot device.

Units run from 0 to 7, so up to eight devices can be attached to the machine. That is a separate limit from the six drive letters: the six is how many partitions may be *in use at once*, drawn from any of the eight devices in any combination. Which units exist depends on the machine's ROM - a ROM that supports only one device ignores the number, so on those machines only unit 0 is meaningful.

Whole devices. A floppy disk has no partition table; the file system starts at the very beginning. Use partition number `0` for that: `MOUNT 1 0 A:` mounts all of unit 1 as `A:`. (`A:` and `B:` are free to use for this, exactly as you would expect, though nothing forces it.)

One drive cannot be unmounted: the one ELF-DOS booted from, because the commands themselves live there. `MOUNT` and `UMOUNT` both refuse.

ELF-DOS reads both FAT16 and FAT12 volumes. FAT12 is what `FORMAT` on another machine will have put on a floppy, and on small flash or RAM disks of roughly 16MB and under; FAT16 is usual on anything larger. You do not have to say which a disk is: ELF-DOS works it out from the disk itself. FAT32 is not supported.

Unformatted partitions. A partition that does not hold a FAT12 or FAT16 file system - a brand new one, or one formatted as FAT32 or by another operating system - is still mounted, but marked as not formatted:

```
C:/> MOUNT 0 2 W:
Mounted partition 2 as W: (not formatted -- FORMAT W: to format it)
```

The letter is reserved for it, and `MOUNT` lists it with `(not formatted)`, but nothing can be read from or written to it (switching to it says `Invalid drive.`) until you `FORMAT` it. See "Formatting a Drive" below. This applies only to partitions: a whole device (partition `0`) must already hold a file system to be mounted, because without a partition table ELF-DOS has no way to know its size.

Changing a floppy. Nothing tells ELF-DOS that you have swapped a disk, and it remembers the old one's layout, so writing to the new disk could damage it. After changing a disk, mount it again before using it:

```
C:/> MOUNT 1 0 A:
Mounted partition 0 as A:
```

That re-reads the new disk and starts you at its root directory. Do it at the prompt, with no program running, and the old disk is left complete: ELF-DOS never leaves a write half-finished once a command has ended.

Formatting a Drive

`FORMAT` writes a new, empty FAT16 file system onto a mounted drive. It works like `FORMAT` on MS-DOS, and like it, it erases everything on the drive. The drive must be mounted first (formatted or not):

```
C:/> FORMAT W:

WARNING, ALL DATA ON NON-REMOVABLE DISK
DRIVE W: WILL BE LOST!
Proceed with Format (Y/N)?Y

Formatting 536,870,912 bytes
Format complete.

Volume label (11 characters, ENTER for none)? PROJECTS

  536,707,072 bytes total disk space
  536,707,072 bytes available on disk

       16,384 bytes in each allocation unit.
       32,758 allocation units available on disk.

Volume Serial Number is 07EA-0A1C
```

If the drive already has a volume label, `FORMAT` first asks you to type it, and stops with `Invalid Volume ID` if you get it wrong. That is your chance to notice you typed the wrong letter. Letters may be typed in either case.

`FORMAT` normally takes only seconds, because it writes just the parts of the drive that describe the file system. **`FORMAT W: -S`** (surface scan) first reads every sector of the drive, showing its progress as a percentage, and sets aside any part that cannot be read (reported as `bytes in bad sectors`). It is slow: about 8 seconds per megabyte on an SD card with a 4 MHz clock, so roughly 4 minutes for 32 MB, over an hour for 512 MB, and about 9 hours for the largest partition (4,095 MB). A slower clock takes longer in proportion - more than twice as long at 1.79 MHz. It is rarely worth it on flash cards, which seldom report a read error. This is the reverse of MS-DOS, where the scan was the default and `/Q` skipped it. `-Q` is still accepted, and does nothing unless it comes after `-S`.

`-V:label` sets the volume label without asking; `-V:` on its own means no label. A label is at most 11 characters and may not contain `* ? / \ | . , ; : + = < > [ ] "`.

`FORMAT` refuses to format:

- the drive ELF-DOS booted from, since the commands themselves live there;
- a partition that is also mounted under another letter - `UMOUNT` the other letter first;
- a partition smaller than about 2MB (4,150 sectors) or larger than about 4GB (8,387,744 sectors), the limits of FAT16;
- a partition that overlaps the space at the start of the device where the ELF-DOS system itself is installed.

The size of each allocation unit depends on the size of the partition, from 512 bytes on the smallest partitions up to 65,536 bytes on the largest. Nothing is written until you answer `Y`, and the disk does not look formatted to `MOUNT` until the very last step. A format that is interrupted part way leaves the drive unformatted, never half-formatted. `FORMAT` returns an exit code of 0 when it succeeds, 5 if you answered `N`, and 1 for any other failure.

Partitioning a Disk

`FDISK` creates, deletes and lists the partitions on a device, like `FDISK` on MS-DOS. Type `FDISK` for the device ELF-DOS booted from, or `FDISK 1` for unit 1, and choose from its menu:

```
ELF-DOS Fixed Disk Setup Program

Current unit: 1 (64 MB)

1. Create a FAT16 partition
2. Set active partition
3. Delete partition
4. Display partition information
5. Change current unit

Enter choice, or Q to quit: 4

Unit 1:
Partition Status Type          Start LBA       Size  Mounted
    1      A    FAT16             2,048      20 MB  W:
    2           FAT16            43,008      32 MB
```

```
Total disk space is 64 MB; largest free block 11 MB.
```

Creating a partition asks how big to make it, in MB or as a percentage of the whole disk (`50%`); pressing Enter takes the largest size that will fit. It goes at the start of the largest free block. A partition may be from 3 MB to 4,095 MB, the range `FORMAT` can work with. A disk with no partition table is given one first (FDISK asks). A new partition is not usable until you `MOUNT` and `FORMAT` it.

Deleting a partition asks you to confirm, and refuses while the partition is mounted - `UMOUNT` it first. That also protects the partition ELF-DOS itself is running from.

A few things work differently from MS-DOS:

- A device has at most four partitions, all primary. ELF-DOS does not use extended partitions or logical drives.
- The first megabyte of every device is kept free for the ELF-DOS system that `SYS` installs, so the first partition starts at sector 2,048. Partitions start and end on whole megabytes.
- The active flag is kept for other systems' sake; ELF-DOS itself starts up the same way whichever partition is active.
- `FDISK` finds a device's size by reading ever further into it until a read fails, so it works with any Elf/OS ROM. If it cannot tell - a device of 8GB or more, or a ROM that never reports reading past the end - it asks you, offering 8GB. **Check that answer**: telling `FDISK` a disk is larger than it really is lets it create partitions that run off its end.

Changes take effect at once: there is no need to restart before mounting a new partition. At the next restart, ELF-DOS mounts the boot device's formatted partitions as `C:` to `F:` as usual.

Building a Bootable Disk

With a second device attached, `FDISK`, `FORMAT` and `SYS` can build a complete ELF-DOS disk without any other computer. If unit 1 is the new disk, and `mbr.bin` and `kernel-full.bin` are on the current drive:

```
C:/> FDISK 1                    (create partition 1, then Q)
C:/> SYS 1 mbr.bin              (the start-up code in sector 0)
C:/> SYS 1 kernel-full.bin      (ELF-DOS itself)
C:/> MOUNT 1 1 W:
C:/> FORMAT W:
C:/> MD W:/bin
C:/> XCOPY /bin W:/bin -y       (the commands, including the shell)
```

Copy `/cfg` and anything else you want the same way. The new disk starts up with its first partition as `C:`.

Typing Commands

A command line is the command's name, followed by any arguments it needs, separated by spaces. For example:

```
TYPE readme.txt
```

File names. A file or directory name can be typed on its own (meaning "in the current directory"), or with a path in front of it:

- `/cfg/env.dat` - an absolute path, starting from the root of the current drive.
- `D:/cfg/env.dat` - an absolute path on a specific drive.
- `../backup` or `cfg/env.dat` - a path relative to the current directory.

Quoting and spaces. If a file name contains a space, put the whole path in quotes so it is treated as one argument instead of two:

```
TYPE "my notes.txt"
```

Arguments with special characters may be delimited by double quotes (`"..."`) or single quotes (`'...'`). The difference is what happens to a backslash inside them. Inside double quotes, and outside of quotes entirely, a backslash followed by a character is taken literally - `\"` becomes a plain `"`, `\\` becomes a single `\`, and `\ ` becomes a plain space that will not split the argument. Inside single quotes, nothing is special; every character between the quotes, including a backslash or a double quote, is used exactly as typed.

Wildcards. The characters `*` and `?` can be used in a file name to match more than one file, the same as in MS-DOS. `*` matches any run of characters; `?` matches any one character. Wildcards are understood by `DIR`, `LS`, `COPY`, `MOVE`, `DEL`, `ATTRIB`, `XCOPY`, and `YS`. If a wildcard does not match anything, the command is given the text exactly as typed instead. As in MS-DOS, a wildcard skips hidden and system files, so `DEL *.*` leaves them alone; `ATTRIB` and `LS -a` still match them, and a hidden file named in full is always found.

How ELF-DOS finds a command. When you type a name on its own, ELF-DOS looks for it in three places, in this order, and runs the first match:

1. the current directory;
2. `/bin` on the drive it booted from, where the system commands live;
3. each directory listed in the `PATH` environment variable, in order.

```
C:/> EXPORT PATH=D:/tools;E:/games
```

Separate directories with a semicolon. `PATH` starts out unset, which simply means only the current directory and the system commands are searched. Because the system commands come before `PATH`, a mistake in `PATH` can never leave you unable to type `EXPORT` to fix it, and a `/bin` on some other disk you have mounted cannot quietly replace `COPY` or `DEL` with its own version. A program in the *current* directory, however, is found first, as in MS-DOS.

You do not have to type `.bat`. If the name you type has no extension, ELF-DOS tries it in each directory first exactly as typed (programs are stored with no extension), then with `.bat`, before moving on to the

next directory. So a batch file named `backup.bat` can be run by typing just `backup`. If you do type an extension, only that exact name is looked for.

To run a program without the search, give a path to it - any name containing a `/`:

```
C:/sub> ./mygame
C:/sub> ../tools/build
```

That runs exactly the file you named (still trying `.bat` if you left the extension off) and nothing else. The root directory has no `.` entry, so at the root write `/mygame` rather than `./mygame`.

If a command name is not found anywhere, ELF-DOS says `Bad command or file name`. That message is about the command itself; a message like `File not found.` comes from a command that ran and could not find one of *its* arguments.

Substitutions. A few special sequences are replaced with a value before a command line runs:

Type this	You get
`%ERRORLEVEL%`	The exit code of the command that just finished.
`%0` through `%9`	Inside a batch file, the file's own command-line arguments (`%0` is the batch file's own name).
`$NAME` or `${NAME}`	The value of environment variable `NAME` (see "Environment Variables" below). An unset variable is replaced with nothing.

These are only replaced outside of quotes, or inside double quotes. Put text in single quotes if you want a `%` or `$` left alone.

Editing the Command Line

While typing a command, you can move around and fix mistakes without retyping the whole line:

Key	Does this
Left / Right arrow, or Ctrl-B / Ctrl-F	Move the cursor one character.
Home / End, or Ctrl-A / Ctrl-E	Jump to the start / end of the line.
Backspace	Delete the character before the cursor.
Delete, or Ctrl-D	Delete the character under the cursor.
Up / Down arrow	Recall an earlier command, or step back to a later one.

These same keys also work inside the `EDLIN` text editor. A line longer than the screen is wide continues on the next row, and the keys work across rows the same way; the editor learns the width from `TERMSIZE` (below), and assumes 80 columns until `TERMSIZE` has run.

Every command you type is saved to a history file (`history.dat`, on the boot drive), and this history is what the Up and Down arrows recall. The history file trims itself automatically once it grows large, so it

never needs to be cleaned out by hand.

Redirecting Input and Output

A command's output can be sent to a file instead of the screen, and a command that reads input can be given a file to read from instead of the keyboard:

```
DIR > listing.txt          Send DIR's output to listing.txt (replacing it)
DIR >> listing.txt         Same, but add to the end of the existing file
SOMECOMMAND < input.txt    Read input from input.txt instead of the
keyboard
```

The special name `NUL` can be used on either side. `> NUL` throws output away; `< NUL` acts like an empty file that ends immediately. This works the same way it does in MS-DOS and Unix.

The `>`, `>>`, and `<` symbols can appear anywhere on the command line, and a space before the file name is optional (`>file` and `> file` both work). Redirection also works inside batch files.

The Command Reference

In the tables below, an argument in `<angle brackets>` is required; one in `[square brackets]` is optional. `...` means "one or more, repeated."

Looking at files and directories

Command	Usage	What it does
`DIR`	`DIR [path...]`	Lists a directory. With no arguments, lists the current directory. Given one or more files or wildcard matches, shows information about each one instead.
`LS`	`LS [-laF] [path...]`	Lists a directory, Unix-style. Plain columns by default. `-l` gives a detailed, one-line-per-entry listing. `-a` includes hidden files. `-F` marks directories with a trailing `/`. Options can be combined (`-lF`) or separate, and go anywhere on the line.
`CD`	`CD <path>`	Changes the current directory. `CD ..` goes up one level; `CD /` goes to the root.
`PWD`	`PWD`	Prints the full path of the current directory.
`STAT`	`STAT <path>`	Shows a file or directory's type, size, first cluster, and the date and time it was last written.
`ATTRIB`	`ATTRIB [+R|-R] [+H|-H] <path...>`	Shows or changes a file's read-only and hidden attributes. With no flags, shows each file's attributes as two columns, `R` and `H` (or `-` when not set). `+R`/`-R` sets or clears read-only; `+H`/`-H` hides or unhides. Flags can be combined (`ATTRIB +R +H file`). A read-only file can be read and renamed but not changed, overwritten, appended to, or deleted: `DEL`,

Command	Usage	What it does
		`COPY`, `MOVE`, `XCOPY`, and output redirection (`>`/`>>`) report "Access denied." The attribute has no effect on a directory.
`LABEL`	`LABEL [drive:] [text | -d]`	Shows, sets, or removes a drive's volume label. `-d` deletes the label.
`MOUNT`	`MOUNT [unit partition letter:]`	With no arguments, lists the drives in use. Otherwise attaches a partition of block device `unit` (0-7, always required) to a drive letter. Partition `0` means "the whole device, no partition table," for floppies. See "Drives" above.
`UMOUNT`	`UMOUNT <letter:>`	Detaches a drive letter. The boot drive cannot be detached.
`FDISK`	`FDISK [unit]`	Creates, deletes and lists the partitions on block device `unit` (0-7, default: the unit the system booted from), from a menu. See "Partitioning a Disk" above.
`FORMAT`	`FORMAT <letter:> [-S] [-V:label]`	Erases a mounted drive and writes a new, empty FAT16 file system on it. `-S` first reads every sector and sets aside any that fail (slow: about 8 seconds per megabyte at 4 MHz); `-V:label` sets the volume label without asking. See "Formatting a Drive" above.
`CHKDSK`	`CHKDSK [X:] [-f]`	Checks a drive for file system problems - lost clusters, damaged cluster chains, files whose size does not match their data, and damaged directory entries - and prints a summary. Without `-f` nothing is changed. With `-f` it repairs them as MS-DOS `CHKDSK /F` did: it asks `Convert lost chains to files (Y/N)?` and either saves each lost chain as `FILE0000.CHK`, `FILE0001.CHK`, ... in the root directory (names are upper case) or frees the space; ends a damaged chain at its last good cluster; sets a file's size to what its chain holds; and corrects wrong `.` and `..` entries. Cross-linked files are reported but not repaired - copy each one, then delete the originals. Do not redirect the output of `CHKDSK -f`.

Working with files

Command	Usage	What it does
`TYPE`	`TYPE <filename>`	Displays a text file on the screen.
`WHICH`	`WHICH <command> [command...]`	Searches for each command the way the shell does (current directory, the system `/bin`, then `PATH`, trying `.bat` for a name with no extension) and prints the full path of the file it would run, or `<name>: not found`. `IF`, `GOTO`, `REM` and a drive change such as `D:` are reported as shell built-ins.

Command	Usage	What it does
LESS	LESS [-N] <filename>	Pages through a file, forward AND backward, by screen or by line. -N numbers the lines, as in less -N: each line starts with its number, right-justified in 7 columns, and the text keeps the rest of the screen. Paging and scrolling cost nothing extra, but with -N a long jump into a large file (G, a search) has to count the lines it skips -- several times quicker than a numbered jump such as 50000g, but no longer instant. SPACE/F/PgDn = next page, B/PgUp = previous page, g = go to the top, G = go to the end; typing a number first jumps there -- 50g (or 50G; either letter works) goes to line 50, and a number past the end of the file shows the last page, the same as a bare G; Down-arrow/J/Ctrl-N/Ctrl-E = down one line, Up-arrow/K/Ctrl-P/Ctrl-Y = up one line (moving up or back past the point where LESS has any recorded history -- e.g. right after G, g, or a search match -- scans backward through the file for the true previous line/page instead of just stopping); / = search forward (case-sensitive), N = repeat the last search, Q = quit. A search pattern may contain escape sequences -- \\n, \\r, \\t, \\0, \\\\ for a literal backslash, and \\xHH for any byte by hex value (exactly two digits) -- so you can search for tabs, line endings, or arbitrary binary bytes, and a pattern may span a line boundary. A backslash followed by anything else is reported as an error rather than searched for literally. A line wider than the screen (COLUMNS) is cut off at the right edge instead of wrapping; the whole line is still searched. Colour and style codes in the file (ANSI escape sequences, such as MDV writes) are shown as colours and take up no room on the line; any other escape sequence is left out, and so is a carriage return.
HEXDUMP	HEXDUMP [-c] <filename>	Shows a file's raw bytes, 16 to a row: the offset, the bytes in hexadecimal, and the same bytes as text. The rows appear in the same pager as LESS, with the same keys, except that a number before g is a byte offset (typed in decimal, e.g. 65536g) rather than a line number, and a search matches the file's raw bytes, so /\\x00\\xff finds binary data. -c prints every row straight through with no paging, so the output can be redirected to a file: HEXDUMP -c data.bin > dump.txt. Offsets are shown in full for files of any size. Each row is fitted to the screen width (the COLUMNS variable): 16 bytes at 80 columns or when COLUMNS is not set, 12 at 64, and up to 32 on a wide screen. -c uses the same width.
MDV	MDV [-c] [-p] <filename>	Shows a Markdown file formatted for the terminal, using ANSI colours and styles: headings in bold colour, **bold**, *italic*, `code` in yellow, links underlined with the address after them, bullet and numbered lists with their wrapped lines indented, block quotes with a bar down the left, code blocks, rules and table rows.

Command	Usage	What it does
		Paragraphs are re-flowed to the screen width (the `COLUMNS` variable). The output is paged: after each screenful `--More--` waits for a key -- `SPACE` shows the next page, `ENTER` or Down-arrow one more line, `Q` quits. `-c` shows it all without stopping, and paging is off anyway when the output is redirected, so `MDV README.MD > README.ANS` writes a file you can read later with `TYPE` or, scrolling both ways, `LESS`. `-p` leaves out the colours and styles and keeps just the layout. Italic is not shown by every terminal.
`COPY`	`COPY [-y] <source> <destination>`	Copies one file to another name, or one or more files into a directory. `-y` skips the "overwrite?" prompt.
`CMP`	`CMP [-l] [-s] [-b] [-n <count>] [-i <skip>[:<skip2>]] <file1> <file2> [<skip1> [<skip2>]]`	Compares two files byte by byte, like the Unix `cmp`. Identical files print nothing. Otherwise it stops at the first difference and prints `<file1> <file2> differ: byte N, line M` (both counted from 1). `-l` lists every differing byte instead: its number in decimal, then the byte from each file in octal. `-s` prints nothing at all. `-b` adds the differing bytes to either report, each as its octal value and the character itself (`^X` for a control character, `M-` before a byte of 128 or more). `-n <count>` compares at most that many bytes. `-i <skip>` skips that many bytes at the start of both files, and `-i <a>:<b>` skips `<a>` bytes of the first and `<b>` of the second; byte and line numbers then count from the first byte compared. The skips can also be given as one or two numbers after the file names (`CMP a.dat b.dat 100 200`), which replace the `-i` value for that file. The numbers are decimal and may be joined to the flag (`-n100`) or follow it (`-n 100`). If one file is just the start of the other, it prints `cmp: EOF on <the shorter file>` and where that file ended. The exit code is 0 when the files are the same, 1 when they differ and 2 when something went wrong (a file that cannot be opened, bad arguments), so a batch file can test it: `CMP -s a.dat b.dat` then `IF %ERRORLEVEL%==0 ECHO same`. `-l` and `-s` cannot be used together.
`MOVE`	`MOVE <source> <destination>`	Moves or renames one or more files, the same way `COPY` takes its arguments.
`REN`	`REN <path> <newname>`	Renames a file or directory. It must stay in the same directory — use `MOVE` to move it elsewhere.
`DEL`	`DEL <filename...>`	Deletes one or more files. It will not delete a directory — use `RD` for that.
`TOUCH`	`TOUCH <filename...>`	Updates a file's last-modified time to right now, without changing its contents. It will not create a new file.

Command	Usage	What it does
XCOPY	XCOPY [-s] [-e] [-y] [-d] [-c] [-r] <source> <destination>	Copies files, and optionally whole directory trees. -s includes subdirectories. -e keeps empty subdirectories that get copied along the way. -d only copies files that are newer than what's already at the destination. -c keeps going if one file fails, instead of stopping. -y skips overwrite prompts. -r overwrites read-only files at the destination (still asking first unless -y); without it they are reported as "Access denied." The copy is not read-only afterwards.

Directories

Command	Usage	What it does
MD	MD <path>	Makes a new, empty directory. The directory above it must already exist.
RD	RD <path>	Removes an empty directory. This will fail if the directory has files in it.

Environment Variables

Environment variables are simple NAME=VALUE pairs that programs and batch files can read. They are persistent across reboots.

Command	Usage	What it does
EXPORT	EXPORT [name[=value]...]	With no arguments, lists every variable. EXPORT NAME=VALUE sets one; EXPORT NAME sets the variable to an empty string. Use UNSET to delete a variable.
PRINTENV	PRINTENV [name...]	With no arguments, lists every variable and its value. Given one or more names, prints just those values.
UNSET	UNSET name...	Removes one or more variables.

Two variables describe your screen. ROWS is its height and COLUMNS its width, in characters. LS fits its columns to the width; LESS and HEXDUMP use the height for the page length and cut every line, status line included, to fit the width; HEXDUMP also fits its rows to the width; MDV wraps its text to the width and pages by the height; EDLIN uses the height for its page length.

TERMSIZE asks the terminal for its real size and sets both variables, and also gives the size to the system itself, so a line in AUTOEXEC.BAT is all it takes. For a terminal that does not answer the size request, give the size yourself: TERMSIZE 24 64 sets 24 rows and 64 columns.

Programs take the size TERMSIZE gave the system first, then ROWS/ COLUMNS, then 24 by 80. EXPORT COLUMNS=64 on its own therefore works only until TERMSIZE has run; after that, change the size with TERMSIZE (TERMSIZE 0 0 hands it back to the variables). The command-line editor uses only the size from TERMSIZE, and assumes 80 columns without it, since it cannot afford to read a variable every time you press a key.

Date, time, and the system

Command	Usage	What it does
DATE	DATE [MM/DD/YYYY]	Shows or sets the date.
TIME	TIME [HH:MM[:SS]]	Shows or sets the time. Seconds are optional.
BAUD	BAUD <rate>	Sets the console's baud rate (300 through 57600).
VER	VER	Prints the ELF-DOS version.
REBOOT	REBOOT [unit]	Restarts the computer without turning it off, booting block device unit (0-7, default: the unit ELF-DOS booted from) the way the ROM would, so it can also start an Elf/OS disk. Refuses a unit whose first sector is blank, or an ELF-DOS disk whose kernel is missing.
SYS	SYS [unit] <file | name>	Installs ELF-DOS onto block device unit (0-7, default: the unit the system booted from). Given kernel-full.bin, installs the kernel; given mbr.bin, replaces the boot code in the device's MBR and keeps its partition table. Given a name that is not a file, installs the pair KSAVE writes, name.krn and name.mbr: both are checked before anything is written, and you are asked once. Refuses a device with no partition table, or where the kernel would run into a partition.
KSAVE	KSAVE [unit] [name]	Saves the system installed on block device unit (0-7, default: the unit the system booted from) as two files: name.krn, the kernel, and name.mbr, the MBR's boot code (default name elfdos). They are a matched pair - the boot code knows the kernel's layout - so SYS name puts both back. The partition table is not saved.
CLS	CLS	Clears the screen.

Sending and receiving files

These commands transfer files over the serial port, to and from another computer running matching transfer software.

Command	Usage	What it does
MR	MR [-u|-b] [-v] [<destination>]	Receives one or more files (the host decides how many). No destination given, or an existing directory, receives everything into that directory (or the current one) under each file's own name; any other name saves only the first file received, under that exact name. -v prints per-file progress messages; without it, MR stays silent until its final summary.
MS	MS [-u|-b] [-v] <filename> [filename...]	Sends one or more files, using the MAX protocol. Filenames may contain */? wildcards. -v prints per-file progress messages; without it, MS stays silent until its final summary.

Command	Usage	What it does
YR	YR [-u|-b] [-y] [-v]	Receives one or more files in a batch, using the YMODEM protocol. -y overwrites an existing file without asking -- strongly recommended whenever the console and transfer port might share a wire (see below), since the interactive overwrite prompt otherwise has nowhere else to go. -v prints per-file progress messages; without it, YR stays silent until its final summary.
YS	YS [-u|-b] [-k] [-y] [-v] <filename...>	Sends one or more files in a batch, using the YMODEM protocol. Filenames may contain */? wildcards. -k sends 1024-byte data blocks instead of the default 128-byte ones. -y is accepted for symmetry with YR but does nothing -- YS never writes a local file, so there's nothing for it to protect. -v prints per-file progress messages; without it, YS stays silent until its final summary.

All four use the board's normal console I/O by default, so a transfer follows whatever serial port (or other device) the console is already using, with no port selection needed. If the board has more than one serial port, -u (the disk-board UART) or -b (the onboard, bit- banged port) sends the WHOLE transfer over that specific port instead, regardless of what the console currently is. This is useful for running a transfer on a different port than the one you're typing commands on -- for example, watching debug output on the console while a transfer runs on the other port, or driving a transfer from a separate program instead of a terminal session.

All four default to printing nothing until they're done, since their console and the transfer's own wire are frequently the same physical connection -- progress text interleaved with protocol bytes on a shared line can be misread as part of the transfer itself (a real failure mode: MS's own "Sent <name>." message, if it leaked onto a shared wire, could be misread by the far end as the start of the next chunk's length field). -v opts back into per-file progress messages when the console and transfer port are genuinely separate (or you're willing to accept the risk on a shared one). YR's own overwrite prompt is the one thing -v/quiet-by-default can't cover -- it needs a real answer from a real person, so -y (skip the prompt entirely) is the actual fix for that specific case on a shared wire, not -v.

MR/MS's companion on the host side is max-xfr (max-xfr -s <files...> to send, max-xfr -r [<destination>] to receive) -- matching syntax to MR/MS in each direction. max-xfr also takes a -v (progress) and -d <microseconds> (a small delay before each byte it writes, including its own acks) -- needed on some bit-banged links, since the ELF-DOS side's serial receive has no buffering of its own and can miss a byte that arrives before it's actually polling for one.

Editing text

EDLIN is a line editor, similar to the classic MS-DOS EDLIN:

```
EDLIN [filename]
```

Type `EDLIN` alone to start with an empty file, or `EDLIN filename` to open an existing one. `EDLIN` shows a `*` prompt and works one line at a time. The commands it understands are:

Command	Does this
`L` or `P`	Lists lines. `L n` or `L n1,n2` lists a specific line or range.
A plain number	Shows that line, then lets you type a replacement (press Enter to leave it unchanged).
`I`	Inserts new lines before the current line. Type `.` alone on a line to stop inserting.
`A`	Appends new lines at the end of the file, the same way `I` works.
`D n` or `D n1,n2`	Deletes a line or range of lines.
`S text`	Searches forward for `text`.
`R`, `C`, `M`	Replace, copy, or move lines.
`T filename`	Reads another file in at the current line.
`W [filename]`	Saves the file without exiting.
`E [filename]`	Saves the file and exits.
`Q`	Quits without saving.

Wherever a line number is expected, you can also type `.` for the current line, `$` for the last line, or `#` for one past the last line.

Other small utilities

Command	Usage	What it does
`ARGS`	`ARGS [anything...]`	Prints back each argument on its own line. Useful for checking exactly how the shell split up a command line.
`ECHO`	`ECHO [-n] [text...]`	Prints the given text. `-n` leaves off the final newline. `ECHO ON` and `ECHO OFF` turn batch-file line echoing on or off (see below); `ECHO` alone reports which one is currently in effect.

Writing Batch Files

A batch file is a text file of commands, run one after another, the same way you would type them yourself. Its name must end in `.bat`. You can create one with `EDLIN` or by transferring a text file from another computer. To run it, type its name, exactly like any other command - the `.bat` is optional:

```
C:/> myscript
```

A first batch file

Here is a small batch file that lists a directory and shows the date:

```
DIR /cfg
DATE
```

Save this as `show.bat` and run it by typing `show.bat`. ELF-DOS shows each line as it runs, the same as if you had typed it yourself, so the screen looks like this:

```
C:/> show.bat
DIR /cfg
 ... the directory listing ...
DATE
 ... the current date ...
```

Controlling what gets shown

Seeing every line echoed is often more than you want. `ECHO OFF`, on a line by itself, turns this off for the rest of the file:

```
ECHO OFF
DIR /cfg
DATE
```

Now only the commands' own output appears, not the commands themselves. You can also silence just one line, even with echo on, by putting `@` in front of it:

```
@DIR /cfg
```

A blank line, or a line starting with `REM`, does nothing - use `REM` to leave yourself notes:

```
REM This script backs up the configuration directory.
```

Using arguments

A batch file can take its own arguments, the same way any command does. Inside the file, `%1` is the first argument, `%2` is the second, and so on; `%0` is the batch file's own name. For example, `backup.bat`:

```
ECHO OFF
COPY %1 /backup
ECHO Done.
```

Running `backup.bat readme.txt` copies `readme.txt` into `/backup`, then prints `Done.`.

`%ERRORLEVEL%` works the same way, standing for the exit code of the command that just ran (0 usually means success). You can check it right after a command to see whether it worked:

```
COPY %1 /backup
IF %ERRORLEVEL%==0 ECHO Copy succeeded.
```

Testing conditions with IF

`IF` runs a command only when something is true:

```
IF EXIST %1 ECHO The file exists.
IF NOT EXIST %1 ECHO The file is missing.
```

`IF string1==string2 command` compares two pieces of text instead, and runs `command` only if they match exactly:

```
IF %1==readme.txt ECHO That is the readme.
```

There is no `ELSE`. Write the opposite condition on its own line instead, as shown above with `EXIST`/`NOT EXIST`.

Jumping around with GOTO

`GOTO label` skips ahead or back to a line starting with `:label`. This is how a batch file can skip a step, or loop. Putting it together with `IF`, here is a batch file that copies a file only if it exists, and explains itself either way:

```
ECHO OFF
IF NOT EXIST %1 GOTO missing

COPY %1 /backup
ECHO Copied %1 to /backup.
GOTO end

:missing
ECHO %1 does not exist -- nothing copied.

:end
```

If the label named in a `GOTO` doesn't exist anywhere in the file, ELF-DOS prints `Label not found.` and stops the batch file right there.

A few things to keep in mind

- If a command inside the batch file fails, or isn't found, ELF-DOS shows the usual error message and simply moves on to the next line - it does not stop the whole batch file.
- A batch file cannot run another batch file from inside itself.
- If a file named `autoexec.bat` exists in the root directory of the boot drive, it runs automatically every time the computer starts, before the first prompt is shown - a good place to put anything you want set up every time (setting the date, changing to a working directory, and so on).

A Few Notes About the File System

- ELF-DOS uses FAT16 and FAT12, the same file systems used by early MS-DOS. Long, lowercase file names are supported normally, alongside the traditional eight-character-name-plus-three-character-extension form.
- The root directory of a drive has no `.` or `..` entries. A path starting with `.` cannot be used to refer to the root - use a plain name, or a path starting with `/`, instead.